

Bases de datos no relacionales

Proyecto final

Sistema RAG NoSQL con MongoDB

1. Información General del Proyecto

Los estudiantes desarrollarán un sistema de Recuperación y Generación Aumentada (RAG) utilizando MongoDB como base de datos principal. El sistema aprovechará las capacidades NoSQL de MongoDB para almacenar documentos flexibles, implementar búsqueda vectorial mediante Atlas Vector Search y construir un pipeline RAG completo que integre un LLM accesible mediante API gratuita.

El proyecto combina el modelado flexible de documentos NoSQL con técnicas modernas de procesamiento de lenguaje natural y búsqueda semántica, culminando en un sistema inteligente capaz de responder preguntas complejas sobre una base de conocimiento diversa.

Este semestre se incorpora como componente obligatorio la experimentación con estrategias de chunking, que determina cómo se dividen los textos antes de vectorizarlos y tiene un impacto directo en la calidad del sistema RAG.

2. Objetivos de Aprendizaje

Objetivos Principales

- Diseñar esquemas NoSQL flexibles utilizando estrategias de embedding vs. referencing según el caso de uso.
- Implementar modelado de documentos que optimice consultas frecuentes y minimice operaciones costosas.
- Construir pipelines de agregación complejos usando el Aggregation Framework de MongoDB.
- Configurar y utilizar índices especializados (texto, compuestos, vectoriales).
- Procesar y vectorizar contenido multimodal (texto, imágenes, metadatos).
- Implementar búsqueda híbrida combinando filtros tradicionales con similaridad vectorial.
- Integrar un pipeline RAG completo con LLM gratuito y técnicas de context retrieval.
- Experimentar con distintas estrategias de chunking y analizar su impacto en la calidad del sistema. (NUEVO)

3. Infraestructura

Opción A: MongoDB Atlas

- Cluster gratuito M0 (512 MB).
- Acceso nativo a Atlas Vector Search.
- Atlas Search para búsqueda de texto completo.
- Monitoreo integrado.

Opción B: MongoDB Local

- Docker Compose con MongoDB 7.0+.
- Configuración manual para búsqueda vectorial.
- Mayor control, pero menor funcionalidad integrada.

4. Alcance Técnico

Diseño de Datos NoSQL

Definir las colecciones y su estructura siguiendo las siguientes estrategias:

- Embedded: metadatos pequeños, historial de consultas del usuario.
- Referenced: imágenes grandes, documentos compartidos entre usuarios.
- Híbrido: documento principal con referencias a imágenes, pero embeddings de metadatos.

Ingesta y Limpieza de Datos

- Scripts en Python o Node.js para cargar datos desde JSON/CSV.
- Validación básica de esquema usando MongoDB Schema Validation.

Consultas y Agregación

- Uso de Aggregation Pipeline con \$match, \$project, \$group, \$lookup.
- Si se usa Atlas Search: \$search con operadores text, vectorSearch, compound.

Índices

- Índice compuesto en { "fecha": 1, "idioma": 1 }.
- Índice de texto en "contenido_texto".
- Si se usa Vector Search: índice knnVector en el campo de embeddings.

Funciones/Triggers (Opcional)

- Usar Atlas Triggers para mantener campos derivados o auditoría.

API Mínima

Desarrollar una API con Node.js/Express o Python/FastAPI con los siguientes endpoints:

- POST /search → búsqueda híbrida o vectorial.
- POST /rag → genera respuesta usando contexto de MongoDB + LLM.

Pipeline RAG

- Embeddings: usar modelo como all-MiniLM-L6-v2 para texto.
- Almacenamiento: guardar embeddings en una colección con knnVector.
- Recuperación: usar \$vectorSearch o \$search con filtros por metadatos.
- LLM: integrar con API gratuita (ej. Groq + Llama 3.1).
- Prompt Engineering: incluir contexto recuperado en el prompt.

5. Estrategias de Chunking

Una de las decisiones más críticas en un sistema RAG es cómo dividir los documentos en fragmentos (chunks) antes de vectorizarlos. Distintas estrategias producen resultados muy diferentes según el tipo de contenido y las consultas esperadas. Este semestre, la experimentación con chunking es un componente obligatorio del proyecto.

5.1 Estrategias a implementar y comparar

Estrategia	Descripción	Cuándo es mejor
Fixed-size chunking	Divide el texto en fragmentos de N tokens con solapamiento (overlap). Simple y predecible.	Textos homogéneos, documentos sin estructura semántica clara.
Sentence-aware chunking	Respeta los límites de oraciones. No corta en medio de una frase.	Textos narrativos, artículos, noticias, descripciones.
Semantic chunking	Agrupar oraciones por similitud semántica usando embeddings. Produce chunks temáticamente coherentes.	Documentos técnicos, papers, textos con cambios de tema.

5.2 Parámetros de configuración sugeridos

- Fixed-size: chunk_size = 256 tokens, overlap = 32 tokens.
- Sentence-aware: máximo 5 oraciones por chunk, overlap de 1 oración.
- Semantic: umbral de similitud coseno entre 0.75 y 0.85.

5.3 Integración con MongoDB

Cada chunk debe almacenarse como un documento independiente en la colección de embeddings, incluyendo el campo `estrategia_chunking` para poder comparar estrategias dentro de la misma base de datos:

Ejemplo de documento chunk en MongoDB

```
{
  "_id": ObjectId(...),
  "doc_id": ObjectId("ref al documento padre"),
  "chunk_index": 3,
  "estrategia_chunking": "sentence-aware",
  "chunk_texto": "El contenido del fragmento...",
  "embedding": [0.023, -0.117, ...],    // VECTOR(384)
  "modelo": "all-MiniLM-L6-v2",
  "fecha_ingesta": ISODate(...)
}
```

5.4 Experimento de comparación

Cada grupo debe ejecutar el mismo conjunto de 10 consultas de prueba sobre los tres tipos de chunking y documentar los resultados. El informe debe incluir:

- Tabla comparativa con la calidad percibida de las respuestas por estrategia.
- Análisis del número de chunks generados y su longitud promedio por estrategia.
- Conclusión argumentada sobre la estrategia más adecuada para su dominio de datos.

Criterio de evaluación del experimento de chunking

Se evaluará: (1) que se implementen al menos dos estrategias, (2) que los chunks estén correctamente almacenados en MongoDB con el campo `estrategia_chunking`, y (3) que el informe incluya un análisis comparativo con conclusión justificada.

6. Entregas del Proyecto

Entrega 1: Diseño y Configuración

Entregables

1. Documento de Análisis
 - Universo del discurso y análisis de requerimientos.
 - Justificación de decisiones de modelado NoSQL.
 - Comparación embedding vs. referencing.

2. Diseño de Esquema NoSQL
 - Definición de colecciones con ejemplos de documentos (incluyendo colección de chunks con campo estrategia_chunking).
 - Estrategias de indexing planificadas.
 - Schema validation rules.
3. Configuración de Entorno
 - Cluster MongoDB configurado (Atlas o local).
 - Scripts de inicialización.
 - Conexión verificada desde aplicación.
4. Plan de Experimento de Chunking
 - Definición de las estrategias a comparar y sus parámetros.
 - Conjunto de 10 consultas de prueba seleccionadas.

Entrega 2: Implementación RAG Completa

Entregables

5. Dataset Preparado
 - Mínimo 100 documentos de texto.
 - Mínimo 50 imágenes asociadas.
 - Formato JSON válido para carga.
6. Sistema RAG Funcional
 - Pipeline completo de ingesta con embeddings (mínimo dos estrategias de chunking).
 - API REST con endpoints documentados.
 - Integración con LLM gratuito configurada.
7. Demostración de Consultas
 - 5 consultas de ejemplo con evidencias.
 - Casos de uso texto-texto, imagen-imagen, multimodal.
8. Experimento de Chunking
 - Resultados comparativos de las estrategias implementadas.
 - Análisis de chunks generados (cantidad, longitud promedio).
 - Conclusión argumentada sobre la estrategia óptima para el dominio.
9. Código Fuente Completo
 - Repositorio Git con estructura clara.
 - README con instrucciones de instalación.
 - Scripts de carga y configuración.
10. Informe Final
 - Arquitectura técnica implementada.

- Resultados del experimento de chunking con interpretación.
- Resultados y evaluación del sistema.
- Lecciones aprendidas y recomendaciones.
- Comparación con enfoque relacional.

7. Casos de Prueba Obligatorios (Ejemplo)

1. Búsqueda Semántica: "¿Qué documentos hablan sobre sostenibilidad ambiental?"
2. Filtros Híbridos: "Artículos en inglés sobre tecnología publicados en 2024"
3. Búsqueda Multimodal: "Imágenes similares a esta foto de arquitectura"
4. RAG Complejo: "Explica las principales tendencias en energías renovables según los documentos".
5. Comparativo de Chunking: la misma consulta ejecutada con cada estrategia implementada, registrando diferencias en los chunks recuperados.

8. Evaluación con RAGAS (NOTA EXTRA)

Los grupos que deseen obtener puntos adicionales pueden implementar un módulo de evaluación automática del pipeline RAG usando el framework RAGAS. Esta es una habilidad muy valorada en la industria para medir la calidad de sistemas RAG de forma objetiva y reproducible.

8.1 Métricas disponibles

Métrica	Descripción	Rango
Faithfulness	¿La respuesta es factualmente consistente con el contexto recuperado? No inventa información.	0.0 – 1.0
Answer Relevancy	¿La respuesta es pertinente a la pregunta original? Penaliza respuestas incompletas o fuera de tema.	0.0 – 1.0
Context Recall	¿El contexto recuperado cubre la respuesta esperada? Requiere ground truth manual.	0.0 – 1.0

8.2 Implementación sugerida

- Instalar ragas: `pip install ragas`.
- Preparar un dataset con al menos 20 pares (pregunta, ground_truth).
- Ejecutar la evaluación sobre las respuestas generadas por el LLM.
- Almacenar los scores como documentos en una colección evaluaciones de MongoDB.

Ejemplo de código mínimo

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy,
context_recall

result = evaluate(
    dataset=eval_dataset,
    metrics=[faithfulness, answer_relevancy, context_recall]
)
```

Ejemplo de documento evaluación en MongoDB

```
{ "id_consulta": ObjectId(...), "faithfulness": 0.91,
  "answer_relevancy": 0.87, "context_recall": 0.78,
  "modelo_eval": "ragas-v0.1", "fecha": ISODate(...) }
```

9. Tecnologías Recomendadas

Base de Datos

- MongoDB Atlas (Vector Search incluido) o MongoDB 7.0+ local.
- Compass para exploración visual.

ML y Embeddings

- Texto: sentence-transformers (all-MiniLM-L6-v2).
- Imágenes: OpenCLIP o transformers (clip-vit-base-patch32).
- Multimodal: CLIP para búsquedas texto↔imagen.
- Chunking: langchain.text_splitter para fixed-size y sentence-aware; semantic-chunkers para semántico.

APIs de LLM Gratuitas

- Groq API: Llama 3.1, Mixtral (rápido, cuota generosa).
- Hugging Face Inference API: modelos open-source.
- OpenAI Free Tier: GPT-3.5-turbo (limitado).
- Ollama: local, sin límites de API.

Evaluación (para nota extra)

- `pip install ragas` — framework de evaluación automática para sistemas RAG.